

SPECS: Speciated Evolutionary Circuit Synthesis

Yağız Genç^{1, 2}, Stefan Uhlich¹, Andrea Bonetti¹,
Arun Venkitaraman¹, Chia-Yu Hsieh¹, Lorenzo Servadei^{1, 3}

¹*Sony Group Corporation, Switzerland*

²*EPFL, Switzerland*

³*TU Munich, Germany*

Abstract

We propose SPECS, a genetic algorithm for automated analog circuit synthesis with joint topology and sizing optimization. SPECS is inspired by NeuroEvolution of Augmenting Topologies (NEAT) [1], an evolutionary algorithm originally developed to synthesize neural networks. By reformulating the genome representation and adapting the genetic operators to the analog circuit domain, we successfully transfer the core principles of NEAT to analog circuit synthesis. Circuit-specific wiring constraints are incorporated to ensure valid and physically meaningful designs throughout the evolutionary process, and speciation is used to preserve innovation while maintaining population diversity. We evaluate the proposed method on a set of computational circuit synthesis tasks consisting of square, cube, square root, and cube root functions. Experimental results demonstrate that SPECS outperforms benchmark methods across all tasks in both solution quality and reliability. The synthesized circuits and their schematics are available in the supplementary repository¹.

Keywords

Analog circuit synthesis, Evolutionary algorithm, Genetic algorithm, Topology synthesis, Component sizing, Speciation

1 Introduction

Despite decades of research, analog circuit design remains a difficult and time-consuming process that still depends heavily on expert knowledge and manual tuning [2, 3]. Unlike digital design, where abstraction and automation have enabled highly scalable synthesis and optimization flows [4, 5], analog design is characterized by complex, non-convex search spaces, strong coupling between design variables, and strict physical constraints that have limited the adoption of fully automated design methodologies.

Evolutionary algorithms have long been explored as a promising approach for analog circuit synthesis, since they can optimize simulation-based, non-differentiable objectives while handling both topology choices and continuous component parameters [6, 7]. Nevertheless, realizing this potential in an effective and scalable way remains difficult. Many existing methods either assume fixed circuit topologies [8, 9] or restrict the search to combinations of predefined building blocks [10]. In more flexible search settings, efficiently exploring the solution space while avoiding premature convergence remains a key challenge.

In this paper, we introduce SPECS, a novel genetic algorithm for automated analog circuit synthesis inspired by NeuroEvolution of Augmenting Topologies (NEAT) [1]. We adapt NEAT’s core principles, such as incremental topology growth and speciation, to the analog circuit domain by redefining the genome representation, node and connection semantics, and genetic operators to act

on circuit elements and interconnections. Starting from minimal structures, SPECS simultaneously evolves topology and component parameters. Speciation maintains population diversity and shields novel circuit motifs from premature elimination by more mature topologies, leading to more consistent discovery of high-performing solutions across independent runs.

The main contributions of this paper are:

- A novel genetic algorithm for analog circuit synthesis that jointly evolves circuit topology and component parameters,
- A domain-specific genome representation and genetic operators tailored to analog circuits, incorporating wiring constraints to ensure validity throughout evolution,
- A speciation-based search strategy that protects novel circuit candidates and grows complexity only when beneficial,
- An experimental evaluation on computational circuit synthesis tasks consisting of squaring, cubing, square root, and cube root functions, demonstrating improved solution quality and reliability compared to benchmark methods.

The remainder of this paper is organized as follows. Sec. 2 reviews related work. Sec. 3 describes the proposed algorithm in detail. Sec. 4 presents experimental results and comparisons with benchmark methods. Finally, Sec. 5 summarizes the findings and discusses directions for future work.

2 Related Work

Analog circuit synthesis generally involves two main tasks: (i) determining the circuit topology and (ii) selecting component parameter values. Substantial progress has been made on component sizing when the topology is fixed in advance [11–16], but identifying the correct topology remains a challenging and largely open problem.

Several evolutionary approaches have been proposed for topology synthesis. Early methods evolved both topology and component parameters using genetic algorithms, often incorporating constrained building blocks or subcircuit-aware crossover techniques to enhance the validity of generated designs [17, 18]. Genetic programming has also been applied, demonstrating that topology and sizing can be discovered simultaneously through evolutionary search [19, 20]. Meanwhile, grammar-based strategies decode variable-length chromosomes into circuit netlists, with recent extensions introducing modularity and homology to reduce crossover disruption and improve substructure reuse [21, 22]. Graph-based autoregressive formulations have been explored, generating circuits incrementally as graphs before translating them into SPICE netlists [23]. Netlist-level evolution emerged as an alternative, with genetic operators applied directly to normalized SPICE netlists, eliminating the need for an explicit genome encoding [24].

SPECS differs from these approaches through its circuit-native genome representation and mutation operators that explicitly account for wiring constraints, enabling joint topology and sizing

¹<https://anonymous.4open.science/r/specs-artifact-637A/>

We now describe SPECS, the proposed method for automated analog circuit synthesis. First, the genome representation used to encode analog circuits is introduced. Next, the genetic operators, namely crossover and several mutation operators, are detailed, explaining how they act on a genome while enforcing the wiring constraints required for valid analog circuits. Finally, an overview of the complete algorithm is presented, demonstrating how a population of circuits evolves to solve the analog circuit synthesis task.

In the context of genetic algorithms, a genome encodes a candidate solution to an optimization task in a structured representation that enables the application of genetic operators such as crossover and mutation. A genome consists of a collection of genes, each representing a specific aspect of the solution.

This genome structure is not directly applicable to analog circuits due to fundamental topological differences. In circuits, the equivalent of a node is a component, but connections are not defined directly between components. Instead, they are established between nets and component pins, with each component having multiple pins. Moreover, circuit connections are undirected and weightless; tunable parameters are associated with the components themselves, such as resistance values or transistor dimensions.

The schematic diagram shows a CMOS inverter. A PMOS transistor has its gate connected to net_input_0 (1,0) and its source to net_supply_0 (7,0). An NMOS transistor has its gate connected to net_input_0 (1,0) and its source to 0 (GND). The drains of both transistors are connected to net_output_0 (7,1). A resistor is connected between net_supply_0 (7,0) and net_output_0 (7,1). The PMOS drain is also connected to net_output_0 (7,1). The NMOS drain is connected to net_output_0 (7,1) and its source to 0 (GND).

Component, internal net, and innovation IDs are assigned globally throughout evolution. Whenever a new component is created, it receives the next available component ID, and likewise for new internal nets. For connection genes, the identifying pattern is the pair of the component-pin tuple and the net ID. If a connection between a given component pin and a given net appears for the first time during a run, a new innovation ID is assigned; otherwise, the previously assigned innovation ID is reused. Consequently, identifiers within a genome need not be consecutive, since some IDs may correspond to structures created elsewhere in the population or earlier in evolution. When a connection gene becomes invalid due to certain genetic operators, it is disabled instead of removed, preserving evolutionary history. When aligned according to their innovation IDs, the matching connection genes of two genomes provides a meaningful measure of genome similarity, which is used for speciation (see Sec. 3.4).

SPECS employs three types of genetic operators: (i) crossover, (ii) structural mutations, and (iii) sizing mutations. Structural mutations modify circuit topology, while sizing mutations adjust component parameters without altering connectivity. Unlike NEAT, where structural mutations primarily increase complexity, structural mutations in SPECS consist of constructive (*Add Component* and *Split Net*), simplifying (*Delete Component*), and rearranging (*Rewire*) operators. This is required, as the performance of analog circuits is highly topology-dependent: connections do not carry tunable weights, and component parameters alone cannot compensate for structurally unfavorable designs. To address this, the

operator set of SPECS allows not only controlled topology growth but also revision of previous design decisions.

3.2.1 Crossover. Given two parent genomes, crossover produces a single offspring. The topology of the fitter parent, in terms of the task-specific fitness defined in Sec. 4, is inherited directly, reflecting the observation that structural quality is strongly correlated with fitness in analog circuit synthesis. For components shared by both parents (i.e., with the same component ID), the parameters are inherited from either parent with equal probability. Components present only in the fitter parent pass their parameters unchanged to the offspring. This mechanism preserves structural integrity while enabling parameter-level recombination between parents.

3.2.2 Structural Mutations. Structural mutations modify circuit topology via four operators: *Add Component*, *Delete Component*, *Split Net*, and *Rewire*.

A central requirement for structural mutations is to avoid creating floating nets. An internal net is floating if fewer than two component pins are attached to it. Input, supply and ground nets are floating if no pin is attached to them. Since the experimental test fixture connects the output net to ground through a load resistor (see Sec. 4), an output net is also considered floating only if no pin is attached to it. We call a pin selection *safe* if applying the corresponding operation does not create any floating nets. Thus, whenever pins are sampled in the steps below, they are selected at random from the set of safe choices when possible. If no safe choice exists, they are sampled from the set of all existing pins.

After each structural mutation, cleanup rules are applied if necessary: nets with no remaining pins are removed, and internal nets with a single remaining pin are removed after rewiring that pin to another existing net. This ensures that all resulting circuits remain electrically valid throughout evolution.

Add Component. A component type is sampled at random from the predefined library and its parameters are randomly initialized within valid ranges. A new component ID is assigned and the component is inserted according to one of three randomly sampled modes, which differ in how many existing pins in the genome are detached and reconnected through the new component:

- **Zero-detachment mode:** No existing pin is detached.
- **One-detachment mode:** One existing pin and one pin of the new component are selected. The existing pin is detached from its current net, and the two pins are connected through a newly created internal net.
- **Two-detachment mode:** Two existing pins and two pins of the new component are selected. Each detached existing pin is paired with one selected pin of the new component, and each pair is connected through a newly created internal net, creating two new internal nets.

After this initial insertion step, any remaining pins of the new component are connected to randomly sampled nets present in the genome. As a result, the operator introduces a new component while minimally disrupting the genome’s existing topology.

Delete Component. A randomly selected component gene and all its associated connection genes are removed from the genome. This enables topology simplification by removing components that no

longer contribute to performance, allowing the search to recover from previously suboptimal design decisions.

Split Net. A splittable net is selected at random. A net is considered splittable if moving at least two of its pins to a new internal net does not leave it floating, which requires it to have at least four attached pins if it is internal, and at least three if it is an input, output, supply, or ground net. A new internal net is created, and a random subset of at least two pins of the selected net is reassigned to the new net. This partitions a highly connected electrical node by introducing a new net, enabling alternative current paths and more refined connectivity.

Rewire. A pin is selected at random, detached from its current net, and reconnected to a randomly selected existing net. This operator enables connectivity refinement without changing circuit complexity.

In this framework, detaching a pin means invalidating its connection gene. When a connection gene is invalidated due to a constructive operator, it is disabled (its *Enabled* flag set to False), thereby preserving its evolutionary history. In contrast, simplifying and rearranging operators remove the gene directly, since they undo prior decisions and frequently revisit similar complexity levels, where accumulation of inactive genes would inflate genome size and distort structural similarity metrics.

Beyond the floating-net requirement, SPECS can be easily extended with domain-specific wiring constraints by restricting the set of allowed pins or nets during sampling. For example, to prevent the gate terminal of MOS transistors from being connected to supply or ground, nets of these types can be excluded from the candidate set whenever a gate terminal, identified by pin index, is to be connected. Such constraints require no modifications to the overall algorithm, making SPECS easily adaptable to problem settings with specific electrical requirements.

3.2.3 Sizing Mutations. Two sizing operators adjust component parameters while preserving topology.

Resample Parameters. A component is selected at random and all of its parameters are resampled independently and uniformly within their valid ranges. This mutation enables large parameter-space exploration and allows escape from poor local configurations.

Perturb Parameters. A component is selected at random and each of its parameters is perturbed by resampling uniformly within an interval of width 10% of the full valid range, centered at the current value and clipped to allowable bounds. This mutation enables fine-grained local search around promising parameter configurations.

3.3 Distance between Two Genomes

Let g_1 and g_2 denote two genomes. We define their distance as

$$\delta(g_1, g_2) := \frac{c_1 E + c_2 D}{L} + c_3 \Delta_{\text{param}} + c_4 \Delta_{\text{comp}},$$

where E and D are the numbers of excess and disjoint connection genes, as defined in NEAT via historical innovation IDs, and $L := \max(n_1, n_2)$ with n_i the number of connection genes in genome g_i normalizes for differences in genome size. Specifically, disjoint genes have innovation IDs present in one genome but not the other within the overlapping innovation ID range, while excess genes

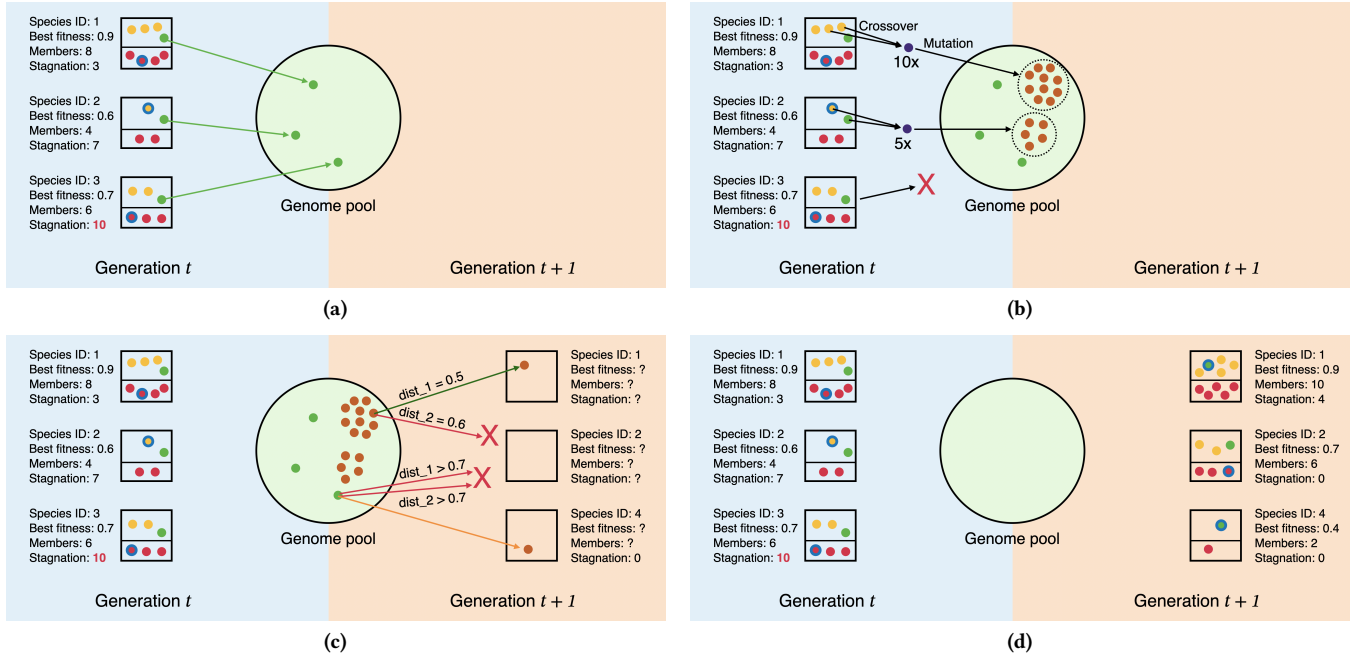


Figure 2: Reproduction mechanism in SPECS. (a) Elite (green) genomes are copied unchanged from each species to the genome pool. (b) Non-stagnated species generate offspring via crossover and mutation and contribute them to the genome pool. Parent selection is restricted to the top fraction (yellow and green), excluding the lower fraction (red) genomes. (c) Each offspring in the genome pool is assigned to the closest species representative (blue border) if the distance is within the compatibility threshold; otherwise it forms a new species. (d) Species statistics and representatives are updated for generation $t + 1$ after fitness evaluations. – Parameters: population size = 18, elite survival fraction = 0.2, parent selection fraction = 0.5, compatibility threshold = 0.7, stagnation threshold = 10, elite size = 1.

have innovation IDs beyond the maximum innovation ID of the other genome. The term Δ_{param} measures the average normalized parameter difference between matching components (those with identical IDs), with continuous parameters normalized by their valid ranges and discrete parameters contributing a unit penalty when mismatched. The term Δ_{comp} captures component-level topological differences as the number of components present in one genome but not the other, normalized by the total number of distinct components across both genomes. The coefficients c_1, c_2, c_3, c_4 control the relative importance of each term. This distance metric plays a key role in the speciation process, as described in Sec. 3.4.

3.4 Proposed Algorithm

We now describe the complete evolutionary procedure used by SPECS. The algorithm follows a generational evolutionary framework with speciation, elitism, crossover, and mutation operators aimed at topology selection and sizing. A schematic illustration of a transition from one generation to the next is provided in Fig. 2.

Initialization. Given a population size N , the initial generation is created by sampling N genomes independently. Each genome contains a single randomly sampled component whose type and parameters are drawn uniformly from their valid ranges, with all pins connected to randomly sampled external nets (input, output, supply, or ground) and no internal nets.

As genomes are created, they are assigned to species. The first genome forms the first species and becomes its representative. For each subsequent genome, its distance to all existing species representatives is computed. If the minimum distance is below the compatibility threshold δ_t , the genome joins the closest species; otherwise, it forms a new species and becomes its representative. After all genomes are assigned, generation 0 is complete.

Species Statistics and Stagnation. At each generation t , every species maintains: (i) its best fitness value, (ii) its number of members, and (iii) a stagnation counter.

The stagnation counter tracks how many consecutive generations have passed without improvement in the species’ best fitness. If this counter exceeds a predefined *stagnation threshold*, the species is considered stagnated and is prevented from generating new offspring in subsequent generations (e.g., Species 3 in Fig. 2(b), where stagnation = 10).

Elitism. To preserve high-performing solutions, a fixed number of elite genomes per non-stagnated species, referred to as the *elite size*, are copied unchanged into the genome pool for the next generation (see Fig. 2(a)).

For stagnated species, instead of a fixed elite size, only a fraction of the top-performing individuals is preserved. Specifically, the number of survivors is given by

$$\max(1, \lfloor \rho_{\text{elite}} \cdot |C| \rfloor),$$

where ρ_{elite} denotes the *elite survival fraction* and $|C|$ is the species size. This ensures that at least one representative of each species is retained, preserving high-quality circuits while gradually suppressing stagnated species.

Offspring Allocation. Offspring allocation determines how many of the remaining $N - K$ individuals each non-stagnated species contributes to the next generation, where K is the total number of elites. Since fitness values may be negative, we use a linear rank-based scheme rather than raw fitness. Reproducing species are ranked by their best fitness; if there are R such species, the highest-ranked species receives weight R , down to 1 for the lowest, and offspring counts are assigned proportionally to these weights (see the highlighted offspring counts for the non-stagnated species in Fig. 2(b)). Unlike NEAT, which allocates offspring counts based on average fitness of species, SPECS decides them based on best fitness. This is motivated by the highly combinatorial and “spiky” nature of the analog circuit search space, where a high average fitness often indicates many genetically similar solutions with limited diversity. In contrast, rewarding species based on their best individual encourages variation and better aligns with the task of finding the best possible solution. Empirically, we observe that many high-performing circuits are discovered by selecting the previously best circuit as parent, further supporting this design choice.

Parent Selection and Reproduction. Within each reproducing species, offspring are generated as follows. For each offspring, we first decide whether to perform crossover according to the *crossover probability*. One parent is selected if crossover is skipped, or two parents if crossover is performed, using linear rank-based weighting. Only the top-performing fraction of individuals within the species, defined by the *parent selection fraction* (ρ_{parent}), are eligible as parents. The selected parents produce an intermediate offspring via crossover; if crossover is skipped, the single parent is treated as the intermediate offspring. Finally, the intermediate offspring may undergo one structural mutation and/or one sizing mutation with predefined probabilities to produce the final offspring. This process repeats until the required $N - K$ offspring are generated.

Parallel-resistor simplification. As a lightweight topology-cleanup step applied to each new offspring, multiple resistors connecting the same pair of nets are replaced by a single equivalent resistor. The resistor with the smallest component ID is retained with its resistance updated to the equivalent parallel resistance, while the remaining resistors and their associated genes are removed. This reduces topological bloat without changing circuit behavior.

Speciation of the New Generation. Once the genome pool for the next generation is complete, each genome is assigned to a species using the same distance-based procedure as in initialization (see Fig. 2(c)). If the minimum distance to any species representative is below the compatibility threshold δ_t , the genome joins the closest species; otherwise, it forms a new species. This is illustrated in Fig. 2(c)–(d), where a genome whose distance to all existing representatives exceeds δ_t forms a new species with ID 4. The representative of each species is then chosen as the medoid, i.e., the genome that minimizes the total distance to all other members.

Fitness Evaluation and Updates. Each genome is evaluated using Ngspice simulations to compute its task-specific fitness. Afterward, species best fitness values, stagnation counters, and representatives are updated (see Fig. 2(d)).

Adaptive Speciation Control. To maintain a target number of species S , the compatibility threshold δ_t is adjusted multiplicatively after each generation: it is multiplied by 1.05 if the number of species exceeds the target, by $1/1.05 \approx 0.952$ if it is below the target, and left unchanged otherwise.

Termination. The evolutionary process continues until the predefined Ngspice simulation budget is exhausted. The circuit represented by the genome with the best fitness encountered during the entire run is returned as the final synthesized circuit.

4 Experiments

4.1 Task Definitions

We evaluate SPECS on a set of computational analog circuit synthesis tasks widely used in the evolutionary circuit design literature [21, 22, 28–32], adopting exactly the same task definitions, simulation setup, and evaluation protocol as in [22].

The circuits implement four mathematical functions of the input voltage: squaring ($V_{\text{out}} = V_{\text{in}}^2$), square root ($V_{\text{out}} = \sqrt{V_{\text{in}}}$), cubing ($V_{\text{out}} = V_{\text{in}}^3$), and cube root ($V_{\text{out}} = \sqrt[3]{V_{\text{in}}}$). For squaring, cubing, and cube root, the input varies within $[-250 \text{ mV}, 250 \text{ mV}]$; for square root, within $[0 \text{ V}, 500 \text{ mV}]$. In each case, $M = 21$ uniformly spaced fitting points are evaluated.

Let $\tilde{V}_i$ and V_i denote the ideal and simulated output voltages at fitting point i , and let $V_{\text{max}} := \max_{1 \leq i \leq M} |\tilde{V}_i|$. A fitting point i is a *hit* if

$$|V_i - \tilde{V}_i| \leq 0.05 V_{\text{max}},$$

and a circuit is a *success* if all $M = 21$ points are hits. The fitness used during evolution is based on the weighted absolute error

$$E := \sum_{i=1}^M w_i |V_i - \tilde{V}_i|, \quad w_i := \begin{cases} 1, & \text{if point } i \text{ is a hit,} \\ 10, & \text{otherwise,} \end{cases}$$

where we define *fitness* $:= 1 - E$. For reporting purposes, we also compute the mean absolute error

$$\text{MAE} := \frac{1}{M} \sum_{i=1}^M |V_i - \tilde{V}_i|.$$

All circuits are evaluated within a fixed test fixture with $\pm 15 \text{ V}$ supplies, a $1 \text{ k}\Omega$ series input resistor, and a $1 \text{ k}\Omega$ output load resistor to ground. The component library consists of default SPICE NPN and PNP BJT models and resistors with values ranging from 1Ω to $10^9 \Omega$, rounded to one decimal place.

To encourage compact circuit designs and prevent uncontrolled topology growth, SPECS enforces an upper bound on the number of components per genome. Once this limit is reached, the *Add Component* mutation is disabled for that genome. For all tasks, the maximum number of components is set to 50.

Circuits are evaluated using Ngspice transient simulations with a voltage ramp input of rise time 0.2 s , consistent with [22].

Table 1: Normalized grid-search $\bar{z}$ scores (lower is better), averaged over the four tasks, for each configuration of parent selection fraction (ρ_{parent}), population size (N), and target species number (S). The best configuration is shown in bold.

ρ_{parent}	N	$S=1$	$S=2$	$S=4$	$S=8$
0.10	50	0.37	1.26	0.49	-0.23
	100	1.26	0.70	-0.31	0.17
	200	0.72	0.54	-0.32	-0.76
	400	0.03	1.08	-0.50	-0.78
0.25	50	-0.02	-0.31	0.28	0.81
	100	-0.17	-0.17	-0.07	-0.65
	200	-0.30	-0.56	-0.22	-0.48
	400	-0.37	-0.29	-1.05	-0.55
0.50	50	0.06	-0.36	-0.10	-0.08
	100	0.23	-0.18	-0.41	-0.12
	200	0.12	-0.20	-0.61	-0.29
	400	0.71	-0.14	-0.28	-0.34
0.65	50	-0.03	-0.32	0.16	0.30
	100	0.25	-0.06	0.36	-0.16
	200	2.18	-0.26	-0.21	0.10
	400	1.14	-0.46	-0.19	-0.44

4.2 Parameter Configuration

The three main runtime hyperparameters of SPECS are *population size* N , *target species number* S , and *parent selection fraction* ρ_{parent} . To select these values, we performed a three-dimensional grid search over population sizes $\{50, 100, 200, 400\}$, target species numbers $\{1, 2, 4, 8\}$, and selection fractions $\{0.1, 0.25, 0.5, 0.65\}$, yielding a total of $4 \times 4 \times 4 = 64$ configurations. Each configuration was evaluated on all four tasks using five independent runs with a budget of 10^6 Ngspice simulations.

For every configuration, we record the median of the minimum MAE achieved across the five runs, yielding one scalar per task. Because the absolute MAE scales differ across tasks, the four per-task scores cannot be averaged directly. We therefore standardize the 64 median-MAE values within each task by z -normalization, and average the four normalized scores to obtain a single comparable score $\bar{z}$ for each configuration. The configuration with the lowest $\bar{z}$ is selected. Table 1 reports the resulting $\bar{z}$ scores for the full grid. The raw per-task median minimum MAE values used to compute the $\bar{z}$ scores are provided in the supplementary repository¹.

The target species number S deserves special attention. Setting $S = 1$ effectively removes the role of speciation, since the population is driven toward a single species, and therefore serves as a useful ablation. Table 1 shows that $S = 1$ yields the highest (worst) $\bar{z}$ scores overall, with averages of 0.39, 0.02, -0.19, and -0.22 for $S = 1, 2, 4$, and 8, respectively, reflecting a consistent improvement with increasing S . This result confirms that speciation is a key component of the proposed method, supporting the intuition that it preserves diversity and avoids premature loss of promising innovations.

The configuration with $N = 400$, $S = 4$, and $\rho_{\text{parent}} = 0.25$ achieved the lowest $\bar{z} = -1.05$ and is therefore used in all experiments reported in this paper. All remaining hyperparameters were fixed to values based on intuition and preliminary experimentation, and were not further optimized. Table 2 summarizes the final parameter settings.

Table 2: Parameter configuration used in all experiments.

Population & Speciation			
Population size (N)	400	Compatibility threshold (δ_t)	0.75
Target species number (S)	4	Stagnation threshold	10
Parent selection frac. (ρ_{parent})	0.25	Elite size	1
Elite survival frac. (ρ_{elite})	0.1		
Reproduction & Global			
Crossover prob.	0.5	Structural mutation prob.	0.8
Sizing mutation prob.	0.9	Max number of comp.	50
Structural Mutation Types			
Add component prob.	0.25	Split net prob.	0.25
Delete component prob.	0.25	Rewire prob.	0.25
Sizing Mutation Types & Add Component Modes			
Resample param. prob.	0.5	Zero-detachment prob.	1/3
Perturb param. prob.	0.5	One-detachment prob.	1/3
		Two-detachment prob.	1/3
Distance Coefficients			
Excess connections (c_1)	0.25	Param. mismatch (c_3)	0.25
Disjoint connections (c_2)	0.25	Comp. mismatch (c_4)	0.25

4.3 Benchmark Methods

We compare SPECS against four benchmark methods from the recent literature that jointly address circuit topology synthesis and component sizing. All methods use the same test fixture, simulation setup, and evaluation metrics, ensuring that observed differences are attributable to the synthesis methods themselves.

GraCo [23] formulates circuit synthesis as an autoregressive graph-construction problem. Circuits are represented as graphs and generated step by step before being translated into SPICE netlists for simulation. The method supports two search strategies: REINFORCE with a leave-one-out baseline (RLOO) and evolution strategies (ES). We use the ES variant, which the original paper reports to yield better results than RLOO on the considered tasks.

SPICEMixer [24] addresses the same joint topology-and-sizing problem but through a substantially different representation. While GraCo generates circuits through an intermediate graph representation, SPICEMixer operates directly on normalized SPICE netlists, applying crossover, mutation, and pruning at the level of netlist lines without the need for a separate handcrafted chromosome encoding. We also benchmark against a further enhanced variant of SPICEMixer, which increases the elite set size from 30 to 100 and uses a richer set of genetic operators, providing broader diversity preservation and finer-grained exploration of the netlist search space. We refer to this variant as SPICEMixer++.

ACID-GE [21] is a grammar-based evolutionary approach that uses grammatical evolution to decode variable-length chromosomes into analog circuit netlists, coupling topology generation and sizing through a grammar-guided genotype-to-phenotype mapping within a single evolutionary process.

ACID-MGE [22] extends ACID-GE by introducing modularity and homology through multi-grammatical evolution. The chromosome is partitioned into separate parts associated with different subproblems, each decoded by its own grammar, reducing the disruptive effect of crossover and improving the reuse of beneficial

Table 3: SPECS vs. GraCo-ES vs. SPICEMixer vs. SPICEMixer++ for the computational circuit tasks. All results are reported over 5 runs with random seeds, each with a budget of 10^6 Ngspice simulations.

Circuit	Algorithm	SR (%)	Mean Hits (%) $\pm$ SD	MAE $\pm$ SD (mV)	Mean BF $\pm$ SD	Max BF	Min MAE (mV)	NCBC
Squaring	GraCo-ES	0	23.8 $\pm$ 9.5	16.86 $\pm$ 1.22	-2.44 $\pm$ 0.26	-1.928	14.41	6
	SPICEMixer	20	52.4 $\pm$ 37.0	9.87 $\pm$ 6.82	-0.90 $\pm$ 1.51	0.974	1.24	27
	SPICEMixer++	20	83.8 $\pm$ 14.3	4.83 $\pm$ 4.26	0.19 $\pm$ 0.88	0.992	0.38	50
	SPECS	80	98.1 $\pm$ 3.8	0.76 $\pm$ 0.51	0.93 $\pm$ 0.07	0.991	0.17	47
Square root	GraCo-ES	0	18.1 $\pm$ 5.6	99.44 $\pm$ 7.77	-19.26 $\pm$ 1.86	-16.086	86.62	3
	SPICEMixer	20	42.9 $\pm$ 29.8	79.96 $\pm$ 46.57	-14.90 $\pm$ 9.96	0.904	4.56	22
	SPICEMixer++	60	83.8 $\pm$ 24.0	25.62 $\pm$ 24.12	-2.66 $\pm$ 4.88	0.906	0.99	241
	SPECS	60	98.1 $\pm$ 2.3	4.44 $\pm$ 4.21	0.56 $\pm$ 0.49	0.989	0.40	47
Cubing	GraCo-ES	0	21.9 $\pm$ 2.3	2.25 $\pm$ 0.01	0.55 $\pm$ 0.00	0.547	2.24	6
	SPICEMixer	20	95.2 $\pm$ 3.0	0.43 $\pm$ 0.15	0.97 $\pm$ 0.02	0.996	0.19	36
	SPICEMixer++	40	90.5 $\pm$ 12.4	0.40 $\pm$ 0.35	0.95 $\pm$ 0.06	0.998	0.10	50
	SPECS	100	100.0 $\pm$ 0.0	0.06 $\pm$ 0.03	1.00 $\pm$ 0.00	0.999	0.03	50
Cube root	GraCo-ES	0	4.8 $\pm$ 0.0	407.00 $\pm$ 0.05	-84.47 $\pm$ 0.01	-84.464	406.97	2
	SPICEMixer	0	78.1 $\pm$ 10.7	23.65 $\pm$ 6.57	-1.99 $\pm$ 1.39	0.196	13.31	32
	SPICEMixer++	40	70.5 $\pm$ 37.4	125.21 $\pm$ 158.40	-24.24 $\pm$ 33.74	0.788	5.36	288
	SPECS	100	100.0 $\pm$ 0.0	3.88 $\pm$ 1.51	0.24 $\pm$ 1.08	0.949	2.14	48

Table 4: SPECS vs. ACID-GE vs. ACID-MGE for the computational circuit tasks. All results are reported over 50 runs with random seeds, each with a budget of 3×10^6 Ngspice simulations.

Circuit	Algorithm	SR (%)	Mean Hits (%) $\pm$ SD	MAE $\pm$ SD (mV)	Mean BF $\pm$ SD	Max BF	Min MAE (mV)	NCBC
Squaring	ACID-GE	52	81.0 $\pm$ 28.5	0.93 $\pm$ 0.41	0.27 $\pm$ 1.17	0.991	–	28
	ACID-MGE	84	97.7 $\pm$ 6.5	0.53 $\pm$ 0.24	0.94 $\pm$ 0.14	0.998	0.08	42
	SPECS	100	100.0 $\pm$ 0.0	0.34 $\pm$ 0.17	0.99 $\pm$ 0.00	0.998	0.09	45
Square root	ACID-GE	44	74.0 $\pm$ 32.3	5.55 $\pm$ 4.32	-5.25 $\pm$ 7.61	0.972	–	35
	ACID-MGE	66	89.0 $\pm$ 24.2	4.01 $\pm$ 2.47	-1.71 $\pm$ 5.78	0.997	0.23	44
	SPECS	90	99.2 $\pm$ 2.6	3.68 $\pm$ 7.62	0.73 $\pm$ 1.22	0.996	0.18	49
Cubing	ACID-GE	76	92.2 $\pm$ 19.3	0.18 $\pm$ 0.07	0.95 $\pm$ 0.12	0.998	–	40
	ACID-MGE	84	95.7 $\pm$ 13.7	0.10 $\pm$ 0.04	0.97 $\pm$ 0.08	0.999	0.05	47
	SPECS	92	99.0 $\pm$ 5.4	0.06 $\pm$ 0.03	1.00 $\pm$ 0.00	0.999	0.02	48
Cube root	ACID-GE	2	11.5 $\pm$ 20.2	10.81 $\pm$ 0.00	-75.95 $\pm$ 23.74	0.773	–	41
	ACID-MGE	22	70.2 $\pm$ 29.4	7.13 $\pm$ 3.70	-12.87 $\pm$ 25.02	0.964	2.04	57
	SPECS	100	100.0 $\pm$ 0.0	5.56 $\pm$ 4.80	0.77 $\pm$ 0.49	0.970	1.41	48

substructures. According to reported results, ACID-MGE improves over ACID-GE in success rate and MAE on the computational circuit tasks, though often at the cost of increased circuit complexity.

Overall, these benchmarks represent distinct paradigms for joint topology-and-sizing optimization: graph-based generation in GraCo-ES, direct netlist-level evolution in SPICEMixer and SPICEMixer++, and grammar-based evolution in ACID-GE and ACID-MGE.

4.4 Results

Table 3 and Table 4 summarize the results. Following [22], SR denotes the success rate, NCBC the number of components in the best circuit, and BF the best fitness.

To provide an initial reference for performance, Table 3 compares SPECS against GraCo-ES, SPICEMixer, and SPICEMixer++ using 5 independent runs with 10^6 Ngspice simulations each, the same budget used for hyperparameter search. GraCo-ES fails to synthesize any successful circuit, consistently collapsing to solutions with very few components (NCBC of 2 to 6) and highly negative mean BF values, indicating premature convergence to structurally degenerate circuits. SPICEMixer and SPICEMixer++ perform substantially better but still fall well short of SPECS across all aggregate metrics

on all tasks. In terms of the best circuits found, SPECS achieves higher Max BF and lower Min MAE on every task, with the only exceptions being squaring and cubing, where it is effectively tied with SPICEMixer++ in Max BF.

Table 4 compares SPECS against ACID-GE and ACID-MGE following the evaluation protocol of [22], where each method is evaluated over 50 independent runs with a budget of 3×10^6 Ngspice simulations, enabling direct comparison with published results. SPECS improves over ACID-GE on all metrics except NCBC, across all tasks. The more demanding comparison is against ACID-MGE, the strongest benchmark method overall, which SPECS still consistently outperforms in all aggregate metrics across all tasks. The largest gap appears on cube root, where SPECS reaches 100% SR against ACID-MGE’s 22%. On squaring, square root, and cubing, the two methods are effectively tied in Max BF, indicating that ACID-MGE can occasionally find circuits of comparable peak quality; nevertheless, SPECS attains lower Min MAE on three tasks of the four. Notably, these gains are not explained by larger circuits: the average NCBC over all four tasks is 47.5 for both methods.

Fig. 3 provides the transient responses of the best circuits found by SPECS and ACID-MGE on all four tasks. Both methods closely

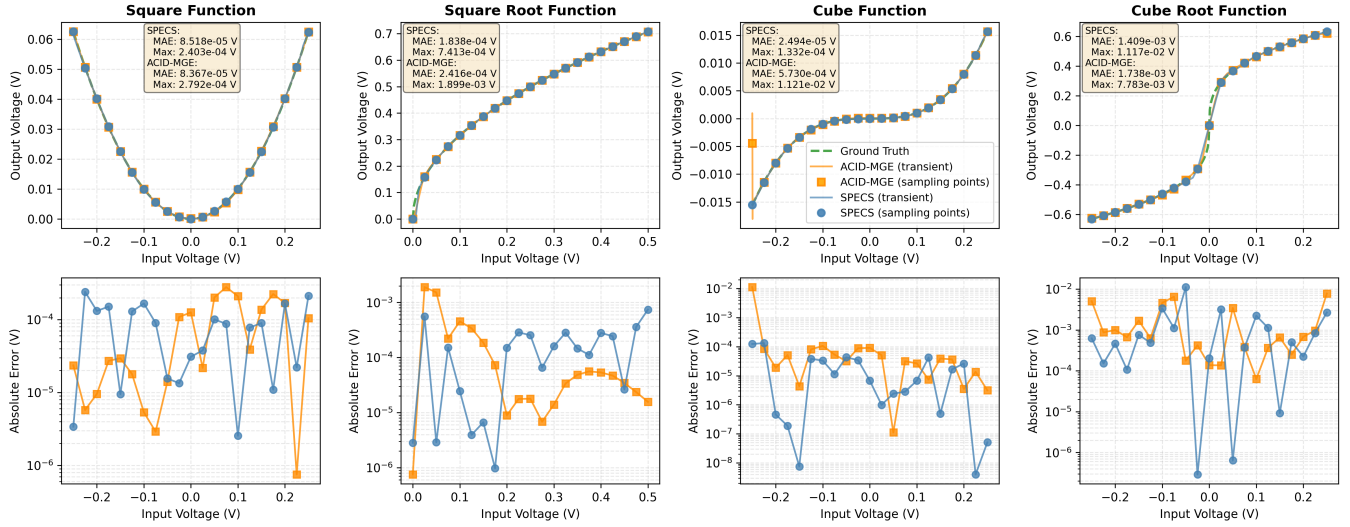


Figure 3: Transient responses of the best circuits found by SPECS and ACID-MGE on the computational circuit synthesis tasks.

approximate the target functions across the full input range. On squaring and square root tasks, the two circuits perform comparably, consistent with the near-tied results in Table 4. On cubing and cube root tasks, however, SPECS achieves a more accurate fit on most fitting points.

The SPICE netlists of the best circuits found by SPECS reported in Table 4, together with their schematics and transient simulation testbenches, are provided in the supplementary repository¹.

The experimental results demonstrate that SPECS achieves the best single-run performance on most tasks and effectively ties the strongest benchmark methods on the rest. Beyond best single-run performance, its most important advantage is reliability: it discovers successful circuits much more consistently across repeated runs than any considered benchmark, as reflected by substantially higher SR and mean hit rates and lower mean MAE. We attribute this to three main factors. First, starting from minimal structures and increasing complexity only through evolutionary improvement ensures that genes present in the population have earned their place, avoiding wasted effort on structurally unjustified combinations. Second, wiring-constraint-aware mutation operators guarantee that all offspring represent electrically valid topologies, avoiding wasted simulations on infeasible candidates. Third, speciation allows novel solutions to compete within their own niches rather than being immediately exposed to more mature topologies, reducing the risk that promising innovations are discarded before they can be refined. This maintains diversity, mitigates premature convergence, and makes the search less likely to become trapped in suboptimal regions of the design space.

5 Conclusion

In this paper, we introduced SPECS, a novel evolutionary algorithm for analog circuit synthesis that jointly searches over circuit topology and component parameters. The method combines a circuit-native genome representation with topology-aware crossover and mutation operators, wiring constraints, and a speciation-based search mechanism that preserves structural innovation while allowing complexity to grow only when beneficial. A key element is the

historical tracking of structural changes through innovation IDs, inspired by NEAT, enabling a meaningful distance metric between genomes and supporting effective speciation throughout search.

We evaluated SPECS on four computational analog circuit synthesis tasks and demonstrated consistent improvements over benchmark methods from graph-based, netlist-level, and grammar-based synthesis paradigms in both best single-run and aggregate performance. The key strength of SPECS lies in three main design principles: minimal initialization that grows complexity only through evolutionary improvement, a domain-specific set of genetic operators that enforce wiring constraints and keep the search within the space of electrically valid circuits, and speciation that protects structural diversity and prevents premature convergence. Together, these make SPECS a reliable and efficient synthesizer that performs strongly not just on the best run, but consistently across independent runs. The fact that the same parameter configuration proved effective across all tasks without extensive tuning further suggests that these principles generalize well beyond the specific problems considered here.

The current evaluation focused on BJTs and resistors, but SPECS is fully compatible with arbitrary component types and fitness functions, making it a general framework for automated analog design. Future work could extend the evaluation to richer component libraries including MOS transistors, capacitors, inductors, and diodes, as well as more application-oriented tasks such as amplifiers, filters, and oscillators. This would put the generalizability of the approach to a broader test and further establish its potential as a widely applicable tool for automated analog circuit synthesis.

Acknowledgements

The authors would like to thank Phuoc Pham (TU Munich) for his valuable assistance in generating the circuit schematics.

References

- [1] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary Computation*, vol. 10, no. 2, pp. 99–127, 2002.
- [2] P. Jespers and B. Murmann, *Systematic Design of Analog CMOS Circuits: Using Pre-Computed Lookup Tables*. Cambridge University Press, 2017. [Online]. Available: <https://books.google.ch/books?id=9GQ3DwAAQBAJ>
- [3] B. Razavi, *Design of Analog CMOS Integrated Circuits*, 2nd ed. McGraw-Hill Education, 2017.
- [4] L. Wang, Y. Chang, and K. Cheng, *Electronic Design Automation: Synthesis, Verification, and Test*, ser. Systems on Silicon. Morgan Kaufmann, 2009. [Online]. Available: <https://books.google.ch/books?id=3XBe7dLb5NEC>
- [5] L. Lavagno, L. Scheffer, and G. Martin, *EDA for IC Implementation, Circuit Design, and Process Technology*, ser. Electronic Design Automation for Integrated Circuits Hdbk. CRC Press, 2018. [Online]. Available: <https://books.google.ch/books?id=MUHMBQAAQBAJ>
- [6] M. Barros, J. Guilherme, and N. Horta, *Analog Circuits and Systems Optimization based on Evolutionary Computation Techniques*, 01 2010, vol. 294.
- [7] E. Tlelo-Cuautle, G.-G. I. M. Duarte-Villaseñor, L. de la Fraga, F.-B. G. G. Reyes Salgado, C. A. Reyes-García, and G. Gómez, "Applications of evolutionary algorithms in the design automation of analog integrated circuits," *Journal of Applied Sciences*, vol. 10, 12 2010.
- [8] B. Liu, Y. Wang, Z. Yu, L. Liu, M. Li, Z. Wang, J. Lu, and F. V. Fernández, "Analog circuit optimization system based on hybrid evolutionary algorithms," *Integration*, vol. 42, no. 2, pp. 137–148, 2009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167926008000126>
- [9] R. Zhou, P. Poehmueller, and Y. Wang, "An analog circuit design and optimization system with rule-guided genetic algorithm," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 12, pp. 5182–5192, 2022.
- [10] T. Dastidar, P. Chakrabarti, and P. Ray, "A synthesis system for analog circuits based on evolutionary search and topological reuse," *IEEE Transactions on Evolutionary Computation*, vol. 9, no. 2, pp. 211–224, 2005.
- [11] K. V. Noren and J. E. Ross, "Analog circuit design using genetic algorithms," 2002. [Online]. Available: <https://api.semanticscholar.org/CorpusID:13454328>
- [12] M. Barari, H. R. Karimi, and F. Razaghian, "Analog circuit design optimization based on evolutionary algorithms," *Mathematical Problems in Engineering*, vol. 2014, no. 1, p. 593684, 2014. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1155/2014/593684>
- [13] Y. Firas, L. Labrak, F. Frantz, R. Daviot, A. Nacer, and I. O'Connor, "A hybrid ga-sqp algorithm for analog circuits sizing," *Circuits and Systems*, vol. 03, pp. 146–152, 01 2012.
- [14] R. Rashid, G. Raghunath, V. Badugu, and N. Nambath, "Performance evaluation of evolutionary algorithms for analog integrated circuit design optimisation," *Microelectronics Journal*, vol. 141, p. 105983, Nov. 2023. [Online]. Available: <http://dx.doi.org/10.1016/j.mejo.2023.105983>
- [15] R. Rashid, K. Krishna, C. P. George, and N. Nambath, "Machine learning driven global optimisation framework for analog circuit design," *Microelectronics Journal*, vol. 151, p. 106362, Sep. 2024. [Online]. Available: <http://dx.doi.org/10.1016/j.mejo.2024.106362>
- [16] M. Kwon, Y. Lee, and I. Song, "Circuit-centric genetic algorithm (cga) for analog and radio-frequency circuit optimization," 2023. [Online]. Available: <https://arxiv.org/abs/2403.17938>
- [17] D. L. Wim Kruiskamp, "Darwin: Cmos opamp synthesis by means of a genetic algorithm," in *32nd Design Automation Conference*, 1995, pp. 433–438.
- [18] A. Das and R. Vemuri, "Gapsys: A ga-based tool for automated passive analog circuit synthesis," in *2007 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2007, pp. 2702–2705.
- [19] J. R. Koza, F. H. Bennett, D. Andre, and M. A. Keane, "Automated design of both the topology and sizing of analog electrical circuits using genetic programming," 1996. [Online]. Available: <https://api.semanticscholar.org/CorpusID:15045531>
- [20] F. Wang, Y. Li, L. Li, and K. Li, "Automated analog circuit design using two-layer genetic programming," *Applied Mathematics and Computation*, vol. 185, no. 2, pp. 1087–1097, 2007, special Issue on Intelligent Computing Theory and Methodology. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0096300306008320>
- [21] F. Castejon and E. Carmona, "Automatic design of analog electronic circuits using grammatical evolution," *Applied Soft Computing*, vol. 62, pp. 1003–1018, 01 2018.
- [22] F. Castejón and E. J. Carmona, "Introducing modularity and homology in grammatical evolution to address the analog electronic circuit design problem," *IEEE Access*, vol. 8, pp. 137 275–137 292, 2020.
- [23] S. Uhlich, A. Bonetti, A. Venkitaraman, A. Momeni, R. Matsuo, C.-Y. Hsieh, E. Ohbuchi, and L. Servadei, "Graco – a graph composer for integrated circuits," 2025. [Online]. Available: <https://arxiv.org/abs/2411.13890>
- [24] S. Uhlich, A. Bonetti, A. Venkitaraman, C.-Y. Hsieh, M. Gürsoy, R. Matsuo, and L. Servadei, "Spicemixer – netlist-level circuit evolution," 06 2025. [Online]. Available: <https://arxiv.org/abs/2506.01497v1>
- [25] T. Harada and E. Alba, "Parallel genetic algorithms: A useful survey," *ACM Computing Surveys*, vol. 53, pp. 1–39, 08 2020.
- [26] D. Sudholt, *Parallel Evolutionary Algorithms*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2015, pp. 929–959. [Online]. Available: https://doi.org/10.1007/978-3-662-43505-2_46
- [27] D. Whitley and S. Rana, "The island model genetic algorithm: On separability, population size and convergence," *Journal of Computing and Information Technology*, vol. 7, 12 1998.
- [28] J. Koza, F. Bennett III, J. Lohn, F. Dunlap, M. Keane, and D. Andre, "Automated synthesis of computational circuits using genetic programming," 09 1997.
- [29] M. Streeter, M. Keane, and J. Koza, "Iterative refinement of computational circuits using genetic programming," 01 2002, pp. 877–884.
- [30] Y. Sapargaliyev and T. Kalganova, "Challenging the evolutionary strategy for synthesis of analogue computational circuits," *JSEA*, vol. 3, pp. 1032–1039, 01 2010.
- [31] —, "Open-ended evolution to discover analogue circuits for beyond conventional applications," *Genetic Programming and Evolvable Machines*, vol. 13, 12 2012.
- [32] L. Muttoni and A. Veiga, "Analog electronic circuit synthesis using simulated annealing and geometric circuit evolution," *Journal of Microwaves, Optoelectronics and Electromagnetic Applications*, vol. 22, pp. 13–32, 03 2023.